prior to using task-specific data. Since HYPERSTEER uses a pre-trained model as a starting point, we postulate that sourcing significantly more data with more concepts of varying and complexity would allow us to test the architecture's limits.

Generating other parameter types Due to compute constraints and our focus on activation steering, we did not explore generating other types of parameter-efficient modulations, including rank-r generalizations such as ReFT (Wu et al., 2024a) or LoRA (Hu et al., 2022) adapters. Such generalizations could potentially be more expressive and allow the hypernetwork to adapt language models to more difficult tasks.

Architecture optimizations HYPERSTEER (Cross Attention) is a parameter-dense transformer model itself. More efficient alternatives could bridge the gap with the dictionary learning baseline and scale up the approach given a limited compute budget.

A.2 Hyperparameter Details

For the `ReFT-r1` baseline, we use the default hyperparameters and settings from AXBENCH. We reduce the number of layer for the cross attention variant to match the total number of parameter with the cross-attention model.

Hyperparameters	*Cross Attention*	*Other variants*
`Gemma-2-2b`		
Batch size	12	6
LR	8e-5	8e-5
N epoch	3	3
Layer	22	26
Cross-attention heads	8	–
`Gemma-2-9b`		
Batch size	4	6
LR	5e-6	5e-6
N epoch	3	3
Layer	34	42
Cross-attention heads	8	–

Table 3: Hyperparameter settings for each variant. For all the unmentioned details such as hidden dimension we use the default configuration of `Gemma-2-2b` and `Gemma-2-9b`.

A.3 Cross-Attention Architecture Details

A token sequence s of length $|s|$ representing the concept for steering is encoded as $\mathbf{h}_0 = \mathrm{Emb}_\Phi(\mathbf{x}) \in \mathbb{R}^{|s|\times d}$. For clarity, we refer to this as the zeroth layer of the residual stream for the hypernetwork $\mathcal{H}_\Phi$.

This precedes N decoder blocks. Each block contains the standard multi-headed self-attention (MHA) feed-forward layer (FFN), and a multi-headed cross-attention module to include information from $\mathrm{LM}_{\mathrm{base}}$. Let $\mathbf{S} \in \mathbb{R}^{|s|\times d}$ and $\mathbf{X}^{(p-1)} \in \mathbb{R}^{|s|\times d}$ be the incoming residual stream. In the p-th block, we compute:

$$\begin{aligned}
\mathbf{X}^{(p)} &:= \mathsf{MHA}\big(\mathbf{Q} = \mathbf{X}^{(\mathbf{p-1})},\ \mathbf{K} = \mathbf{X}^{(\mathbf{p-1})}, \\
&\qquad \mathbf{V} = \mathbf{X}^{(\mathbf{p-1})}\big), \\
\mathbf{X}^{(p)} &:= \mathsf{MHA}\big(\mathbf{Q} = \mathbf{S},\ \mathbf{K} = \mathbf{X}^{(\mathbf{p})}, \\
&\qquad \mathbf{V} = \mathbf{X}^{(p)}\big), \\
\mathbf{X}^{(p)} &:= \mathsf{LayerNorm}\Big(\mathbf{X}^{(p-1)} + \mathsf{FFN}\big(\mathbf{X}^{(p)}\big)\Big).
\end{aligned}$$

We initialize the self-attention MHA blocks from the pre-trained `Gemma-2` base model and the cross attention blocks according to the default PyTorch weight initialization scheme.

A.3.1 Model Size

Our largest HYPERSTEER (cross attention) architecture has 22 modified decoder blocks, and has ≈ 0.998 times the parameters as `Gemma-2-2B`, which has 26 standard decoder blocks. Each cross attention decoder block has ≈ 1.18 times as many parameters as a standard `Gemma-2-2B` decoder block.

A.3.2 Detailed TFLOPs Analysis

We demonstrate that the number of TFLOPs to reach optimal steering performance decays with the number of concepts in the dataset, or steering prompts used. Thus, as we scale up steering data HYPERSTEER becomes an efficient and superior alternative to the supervised dictionary learning baseline for steering language models. We focus our best method, the cross attention architecture, for this analysis.

Let $F_{\mathrm{ReFT}} \approx 666.27 \pm 20.74$ be the TFLOPs required to train a single `ReFT-r1` steering vector, and $\bar{\mathcal{L}}^{\star}_{\mathrm{joint}}$ be the average optimal evaluation loss for computed on `Concept10`. We average over 5 different random seeds as well to obtain this constant. To train HYPERSTEER (cross attention) we construct datasets $\mathcal{D}(c)$ of varying number of concepts (steering prompts) selected in the interval $c \in [10, 16000]$. `Concept10` is *held-in* with respect to $\mathcal{D}(c)$. We train HYPERSTEER on each $\mathcal{D}(c)$ until the eval loss on `Concept10` reaches $\bar{\mathcal{L}}^{\star}_{\mathrm{joint}}$. The TFLOPs per concept $F_{\mathcal{D}(c)}$ for a dataset $\mathcal{D}(c)$, where N^* gradient steps are taken until $\bar{\mathcal{L}}^{\star}_{\mathrm{joint}}$ is

achieved is computed with the following formula:

$$F_{\mathcal{D}(c)} = \frac{N^* \cdot \bar{F}_{\text{step}}}{c}. \quad (5)$$

$\bar{F}_{\text{step}}$ is the average TFLOPs per training step for HYPERSTEER , a local per-training-run statistic with low variance given that the distribution of sequence lengths of both input prompts and steering prompts across examples is observed to be largely uniform. The number of layers is selected to match the total number of parameters with the target model.

We also fit a simple curve to approximate $F_{\mathcal{D}(c)}$, and find that an equation of the form $f(c) = a + b \cdot \exp(dc)$ best fits the curve with $a = 87.7035, b = 1521.1495, c = -0.0034$ and $R^2 = 0.9976$. Clearly, $\lim_{c\to\infty} f(c) = a$ and $a < F_{\text{ReFT}}$, showing that HYPERSTEER is more compute efficient to train when scaling up steering tasks.

A.4 Details on Training Objective

`ReFT-r1` jointly optimizes for steering via causal language modeling loss and concept detection by selecting the top-k sequence-level activations.

$$\mathcal{L}_{\text{joint}}(\text{LM}_\theta) = \quad (6)$$

$$\mathbb{E}_{(\mathbf{x},\mathbf{y})\sim\mathcal{D}}\Big[-\sum_{t=1}^{T} \log P_\theta(y_t \mid \mathbf{x}, y_{<t}) + \lambda \sum_{a_i \notin \text{TopK}(\Psi(\mathbf{h}))} \|a_i\|_1 \Big]. \quad (7)$$

Here,

$$\Psi(h_i) = \text{ReLU}(h_i \cdot \mathbf{w}_{\text{ReFT-R1}}) \in \mathbb{R}^{d\times 1} \quad (8)$$

is a sequence-level concept detection latent. This objective is also used in the regression and SFT variants of HYPERSTEER, when the steering vector is only conditioned on the concept.

Input-conditioned variants do not minimize an additional concept detection loss term, hence do not require an additional inference step to compute the maximal activations on a per-concept basis. The baseline `ReFT-r1`, end-to-end (no in-context learning), and regression variants require this additional step, and the steering heads are modified to generate Δ_s with unit norm. Input-conditioned methods do not normalize Δ_s^x, eliminating this step.

We note that the no-context reconstruction method is trained on ground truth labels that do utilize $\mathcal{L}_{\text{joint}}$, hence evaluation on the regression method requires this additional inference step.

A.5 Concept Dataset Details

Negative samples AXBENCH uses negative examples to enable training and evaluation for concept detection. Since our work only focuses on steering models and not concept detection, we discard this objective and omit negative examples in training and evaluation data for all HYPERSTEER variants. We note that the no-context reconstruction variant indirectly uses these negative samples, for ground truth steering vectors Δ_s^* were trained using the joint objective (7).

Data Ratio For all training datasets, the ratio of base prompts to steering prompts is 72 : 1. During evaluation, the ratio of base prompts to steering prompts is 10 : 1. We keep these parameters consistent with AXBENCH save for the lack of negative examples.

Base prompt data distributions Training base prompts are sampled from a diverse instruction pool of three genres: code, text, and math. The open source datasets `Dolly-15K` (Conover et al., 2023) and `GSM8K` (Cobbe et al., 2021) comprise the instruction data in this pool. We point readers to Sec. I of the AXBENCH appendix for further details. Labels for training data are generated from `gpt-4o-mini`. For evaluation base prompts, we use sample instructions from `AlpacaEval` (Li et al., 2023b) to ensure fairness. These settings and choices are again identical to those of AXBENCH.

A.6 Baseline details

We use prompting, fine-tuning, and activation steering baseliens from AXBENCH. The core comparisons however

Supervised Dictionary Learning `ReFT-r1` is a method proposed by AXBENCH (Wu et al., 2025) to jointly perform the task of concept detection and concept steering using a weakly supervised objective (7). At training time, we train one `ReFT-r1` per concept/steering prompt to populate a dictionary of atoms, with each atom being a learned steering vector.

At training time, the latent is computed from the similarity between the hidden states of the modele model and the learned steering vector:

$$\Psi_{\text{Detect}}^{\text{ReFT-r1}}(h_i) = \text{ReLU}(h_i \cdot \mathbf{w}_{\text{ReFT-r1}}) \quad (9)$$

This latent is then inferred on the evaluation set

Short Description	Full Description
Structured Data Entries	specific identifiers or entries in a structured data format
Personal Identity References	references to personal possessions and identity
Time References	instances of specific time references or moments within the text
Java Interface References	references to Java interfaces and their implementations
Legal Terminology	references to legal terminology and concepts related to law and justice
Mathematical Notation	key phrases related to personal aspirations and career transitions
Proper Nouns	occurrences of mathematical symbols or notation
Employment Contract Terms	proper nouns and names
Object Specifications	phrases related to employment contracts and compensation specifics
Other	references to measurements, specifications, and characteristics of objects

Table 4: Mapping of short concept descriptors to their full labels in held-out `Concept10`.

to determine the final magnitude of the steering vector for each concept at test time:

$$\Delta_s^x = \left(\frac{1}{k} \left\| \text{Top-k}(\Psi_{\text{Detect}}^{\text{ReFT-r1}}(\mathbf{h})) \right\|_1 \right) \mathbf{w}_{\text{ReFT-r1}} \tag{10}$$

Prompt steering This is a strong baseline which is shown in AXBENCH to outperform other steering methods and does not require training. For a given steering prompt s, `gpt-4o-mini` is used to enhance $s \rightarrow s'$ by explicitly instructing the model to include the concept in its response, which is pre-pended to a base prompt x. We sample steered generations from the target LM using this enhanced prompt $s' \oplus x$.

A.7 Additional Experiments

A.7.1 Geometric structure using dimensionality reduction

We also analyze the structure of our high dimensional steering vectors on a held-out evaluation set of steering prompts

A.7.2 Attention Heatmaps

To better understand the interaction between the base prompt x and the steering prompt s in the In Context and Cross Attention HYPERSTEER architectures, we analyze the self-attention and cross-attention heatmaps across layers and heads ($N = 20$ layers).

A key takeaway is that all query (concept) tokens all tend to attend to the same or a few select keys/values (input prompt) tokens with high mass. This trend remains consistent across cross-attention modules from layers 0-19 (9, 11, 12). Each cross-attention module has 8 heads.

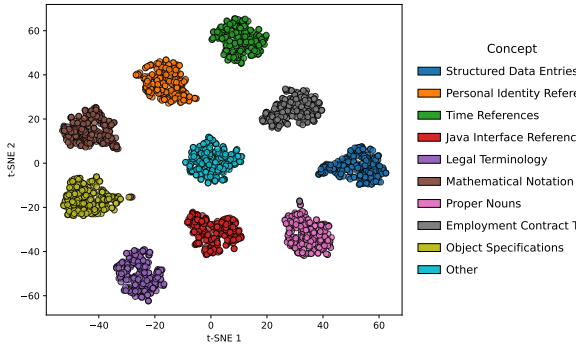

Figure 4: `Concept10` t-SNE analysis of 2500 steering vectors from HYPERSTEER (cross attention), 250 per concept.

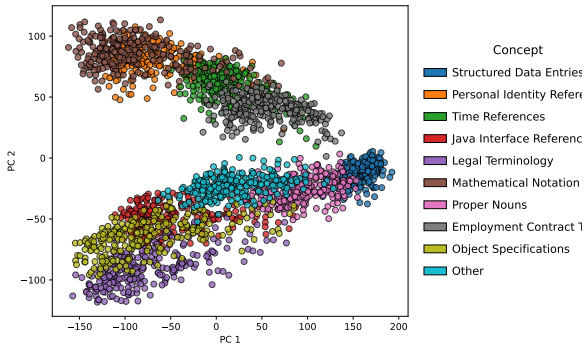

Figure 5: `Concept10` PCA analysis (2 components) 2500 steering vectors from HYPERSTEER (cross attention), 250 per concept.

A.7.3 Data Distribution

A potential issue with our dataset are the use of ground truth labels samples from a stronger "teacher model" `gpt-4o-mini`, whereas `Gemma-2-2B` is a weaker "student" model we seek to adapt. This is evidenced by the right-skewed distribution (see 7 for perplexities computed from base LM when conditioned on the `gpt-4o-mini` output distribution. The perplexity distribution from `Gemma-2-2B` outputs (8) comprises a much smaller range in comparison.